# Robotic Planning Project Report: Autonomous Victim Rescue System

### Sampling-Based and Combinatorial Navigation Approaches

*Diego Oniarti, Luca Podavini, Keith Leoni*

## 1. Introduction and Objectives

### 1.1 Project Overview: The Victim Rescue Challenge

The primary objective of this project is to develop an autonomous navigation stack capable of maximizing a rescue score within a strict time budget ($T_{max}$). The robot operates in a complex environment populated with obstacles and victims, each assigned a specific reward value. The mission is to identify the optimal subset of victims to visit and successfully navigate to the exit gate before the time limit expires. The robot must not only decide which victim to rescue to maximize the total reward but also strictly adhere to non-holonomic motion constraints, ensuring that every planned path is physically executable and without collision.

### 1.2 The Planning Pipeline: From Discrete Graph to Continuous Trajectory

To solve this, we implemented a hierarchical architecture that progressively refines the problem from a global search to a local trajectory.

The process begins by **inflating map obstacles** using the Clipper2 library to ensure safety margins. In the resulting free space, we construct a **Roadmap** using **PRM** (Probabilistic Roadmap) or **ACD** (Approximate Cell Decomposition). To improve the path, we apply a geometric constraint (the 2R rule) to ensure that the connected nodes are far enough apart to prevent the generation of tight $CCC$ maneuvers. We proceed by abstracting the complex roadmap into a simplified one that contains only the key positions (start, goal, and victims) connected by edges whose weights are collected in a **cost matrix** calculated using **Dijkstra's algorithm**. For every edge in the simplified roadmap we keep track of the intermediate nodes in the original one.

The **cost matrix** is feeded into an **Orienteering Problem** solver, which determines the optimal sequence of victims to visit that maximizes the score within the time limit. The **full path** is then reconstructed using the information about intermediate nodes stored during the previous stage. It is refined using **Victim-Aware Smoothing**, which tightens the route in free space while strictly preserving victim visits. Finally, the path is converted into a continuous **Dubins trajectory**.

If the low-level planner cannot generate a valid curve to reach a specific victim (e.g., due to orientation constraints), the system identifies and removes it from the candidate list. Then, the planning restarts

from the orienteering phase, generating a new trajectory that ignores the problematic victim. This process is repeated until a valid, collision-free, and executable trajectory is found.

## 2. Environment Pre-processing

### 2.1 Simulation Environment

The project relies on the *loco_nav* simulation environment, where the *map_pkg* and *obstacles_msgs* packages manage the generation of the world. These packages spawn obstacles, victims, and the exit gate, publishing their coordinates via ROS topics.

To interface with this environment, we implemented an environment handler. On initialization, this handler sets up subscribers to read the map boundaries (*/map_borders*), static obstacles (*/obstacles*), the goal pose (*/gates*), and the victim locations (*/victims*). It merges this data into a single internal representation making sure that every component of the planner has access to the same environment information.

### 2.2 Collision Avoidance: Polygon Offsetting and Safety Margins

To ensure safety, we transform the workspace into a Configuration Space (C-Space) by inflating obstacles. This allows us to consider the robot as a point mass for planning purposes. We utilize the Clipper2 library to perform robust polygon offsetting.

The inflation radius is calculated as $R_{inflated} = R_{robot} + \epsilon_{safety}$, where $R_{robot}$ is the physical radius of the vehicle and $\epsilon_{safety}$ is an additional margin to account for tracking errors. The inflation logic is direction-sensitive: obstacles are inflated outwards, while the map borders are inflated inwards to constrain the robot within the valid area.

By computing these inflated polygons (C-Polygons) during the pre-processing phase, we eliminate the need for repeated radius checks during the search phases, significantly speeding up collision checks.

### 2.3 Collision Detection with Ray Casting

To improve performance we use different techniques to determine collision detection. For circular obstacles we simply check if the robot lies within the obstacle radius, while for polygonal obstacles we use a two-step process. First we exclude far away obstacles using their **AABB** (Axis-Aligned Bounding Box), and for those that don't pass this test we use the more expensive **Ray Casting Algorithm**. The algorithm projects a semi-infinite ray from the point in a fixed direction and it counts how many times this ray intersects the edges of the polygon. If the number is odd, the point is inside the obstacle (collision), if the number is even, the point is outside (safe).

## 3. Global Path Planning: Roadmap Approaches

### 3.1 Probabilistic Roadmaps: PRM and RRT*

In the context of robot planning, the choice of the roadmap planner algorithm significantly impacts efficiency. A popular algorithm is **PRM** (Probabilistic Roadmap), a multi-query planner that constructs a global graph of the environment's free space before starting the mission, so offline. RRT* is a single-query algorithm that grows a tree from a starting configuration toward a specific goal. While RRT* is effective for reaching a single target, it is less efficient for multi-objective missions like victim rescue, as it requires the construction of a new tree for every pair of points (considering start, goal and victims).

In the PRM the nodes of interest are added to the nodes sampled in the free space before connecting them with edges to obtain the final graph, so the roadmap. This is saved in order to be able to query it to find paths between each node pair, such as from the start node to a victim node, without the need to construct a new map every time. The PRM approach was selected over RRT* because its multi-query capability aligns better with the project requirements.

### 3.2 Combinatorial Roadmaps: Exact vs Approximate Cell Decomposition

We chose **ACD** (Approximate Cell Decomposition) as our combinatorial planning method because it represents a good deterministic counterpart to PRM. The roadmaps generated by ACD are qualitatively similar to those produced by PRM, as both describe the free space using a graph of sampled or representative nodes connected through collision-free paths. Because of this similarity, pathfinding algorithms and graph-based planning strategies designed for PRM roadmaps can be applied to ACD ones and vice versa with minimal changes and comparable behavior.

Compared to ECD (Exact Cell Decomposition), ACD avoids excessive subdivision in the presence of inflated obstacles and rounded geometry, where ECD tends to fragment the space unnecessarily. The adaptive subdivision process of ACD refines the environment only near obstacles and boundaries, while leaving large empty regions coarsely represented. This results in a roadmap with more nodes and connections in constrained areas, improving connectivity and increasing the likelihood of finding feasible and efficient paths, without introducing unnecessary complexity in open space.

Nodes representing the victims' locations, the starting point of the robot, and the exit gate are added to the roadmap. A simple approach would be to only connect these points to the node representing the quad in the tree that contains them, but this runs the risk of creating links that are too short with respect to the robot's turning radius (see 3.3). Hence, a number of additional links are created with more of the nearby nodes for safety.

### 3.3 Roadmap Construction

To connect the nodes of the roadmap a criteria based on distance was implemented. In particular, each node tries to connect to its neighbours only if they are at a minimum distance of $2R$, where $R$ is the minimum steering radius of the robot. This is not the real minimum steering radius, but a bigger one considered in the simulation to avoid getting too close to the robot limits and obtain an unstable behaviour in the simulation. This was done to avoid CCC manoeuvres in the Dubins path and achieve faster and cleaner results. A maximum distance was also fixed to avoid connecting too many nodes. If

an edge following these criteria did not collide with any of the obstacles in the environment it was added to the roadmap.

## 4. Solving the Orienteering Problem

### 4.1 Roadmap Abstraction with All-Pairs Shortest Path (APSP)

To solve the navigation problem, the system must determine the cost between each possible combination of nodes of interest. This is achieved through an All-Pairs Shortest Path (APSP) analysis, in which the Dijkstra algorithm is run to find the shortest possible distance between each pair of special nodes in the roadmap. The results are collected in a **cost matrix.**

Using information on shortest paths, a simplified roadmap is constructed containing only special nodes, connected by edges whose weight is determined by the cost matrix. For every edge in the simplified roadmap we keep track of the intermediate nodes in the original one. This allows the orienteering problem to be solved on a small-dimensional graph, avoiding complex optimization algorithms.

### 4.2 The Orienteering Problem

After computing the simplified roadmap, the problem becomes to select the optimal subset of victims to visit and to determine their rescue order. This is modeled as an Orienteering Problem (OP), a variation of the Traveling Salesperson Problem (TSP) where the goal is to maximize the total score collected from visited nodes (victims) within a strict time or distance budget ($T_{max}$). Unlike TSP, visiting every node is not mandatory, the planner must strategically prioritize high-value targets reachable within the constraint. To solve this, the planner implements a two-phase approach, consisting of **Greedy Insertion** followed by **2-Opt Local Search**.

The algorithm initializes a path directly from the start to the end node. It then iteratively evaluates all unvisited victims. For each candidate node, the planner calculates an efficiency metric, defined as the ratio of the victim's score to the distance required to add it to the path ($Score/\Delta Distance$). The candidate with the highest efficiency that fits within the remaining time budget ($T_{max}$) is inserted into the path at the position that minimizes the cost. This process repeats until no further nodes can be added without violating the time constraint. Once the greedy path is constructed, the **2-Opt local search** algorithm is applied to optimize the sequence. This step checks if reversing the visit order of any two nodes reduces the total path length.

### 4.3 Path Reconstruction and Trajectory Generation

Once the optimal sequence of high-level waypoints (e.g., Start $\rightarrow$ Victim A $\rightarrow$ Victim B $\rightarrow$ End) is determined, the planner must convert this abstract sequence back into a detailed, navigable trajectory. This is done thanks to the saved intermediate nodes (Section 4.1). The system reconstructs the dense node-by-node route between each consecutive pair of waypoints in the optimal sequence.

This results in a continuous path through the original roadmap that follows the optimal victim order found solving the orienteering problem.

## 5. Local Motion Planning and Feasibility

### 5.1 Path Smoothing

After the Orienteering problem has determined the sequence of victims, we apply Path Shortcutting. This technique iteratively attempts to connect non-adjacent waypoints with straight lines, removing intermediate nodes. However, we have an additional constraint: victims cannot be excluded from the path during the shortcutting, as this would defeat the main purpose of the planning process. This constraint ensures that the final trajectory remains smooth and efficient while guaranteeing that every selected target is visited.

### 5.2 Trajectory Generation: Implementing Dubins Curves for Non-Holonomic Constraints

While the high-level planner produces a sequence of waypoints in $\mathbb{R}^2$, the vehicle is subject to non-holonomic constraints: since it operates in the configuration space (position and orientation) and it has limited steering capabilities, it cannot for example turn on the spot or translate laterally. To address this problem, the system transforms the poly-line roadmap path into a continuous Dubins trajectory consisting of maximal curvature arcs and straight line segments.

To ensure continuity of orientation between waypoints, the planner does not solve for each segment independently. Instead, it uses a Dynamic Programming (DP) approach over a discretized orientation space. For each intermediate waypoint $p_i$, the orientation space $[0, 2\pi)$ is discretized into a set of $K$ angles, denoted as $\Theta_i = \{\theta_{i,1}, \theta_{i,2}, \ldots, \theta_{i,K}\}$. The start and goal orientations are fixed.

The DP algorithm optimizes a cost function $J(i, \theta)$ representing the minimum traversal distance from the start node to waypoint $p_i$ arriving with orientation $\theta$. The recurrence relation is defined as:

$$J(i, \theta) = \min_{\phi \in \Theta_{i-1}} \left[ J(i-1, \phi) + \mathcal{L}_{\text{Dubins}}((p_{i-1}, \phi), (p_i, \theta)) \right]$$

where $\mathcal{L}_{\text{Dubins}}$ computes the length of the shortest optimal curve connecting two configurations. The algorithm iterates from $i = 1$ to $n$, populating the $J$. If $J(n, \theta_{goal}) < \infty$, the optimal sequence of orientations is reconstructed by backtracking from the goal state, producing a smooth trajectory that respects the entry and exit angle at each node and takes into account the robot's turning radius.

### 5.3 Final Path Validation

The planner calculates the shortest distance based on straight lines between nodes in a graph (roadmap), but the robot follows curves with a minimum radius due to its limited steering ability. To maintain the simplicity of a decoupled approach while ensuring a valid Dubins path, the algorithm generates the simplified roadmap only once, computes the optimal sequence of victims, and attempts to generate a valid Dubins trajectory. If a segment is found to be kinematically infeasible (the path

collides with obstacles or the map borders), the victim causing the error is identified and removed from the candidate list. A new optimal route is calculated using the orienteering solver on the remaining victims until the final path is valid.

This approach is possible by assuming that the only points of failure are the victims. This is a reasonable simplification if we consider how the optimal path is calculated using the 2R rule and then path shortcutting. The former avoids that nodes in the original roadmap are too close together, and the latter tries to create the straightest paths possible while avoiding obstacles. It is therefore very unlikely that a collision will occur halfway along an optimal path; it is much more likely that it will occur near the victims, which are the only nodes unaffected by shortcutting.

## 6. Implementation Challenges and Difficulties

### 6.1 Managing Linear Roadmap Assumptions vs. Curved Execution

The most major challenges we faced was the decoupling between the linear, euclidean roadmap and the curved Dubins trajectories. The high-level planner often outputs a path that satisfies the distance cost and the environment limits but that is impossible for a non-holonomic vehicle in the real world. These paths force the robot into configurations where it must perform sharp turns immediately after obstacles, leading to the feasibility failures described in Section 5.3.

### 6.2 Control Law Refinement and Numerical Stability

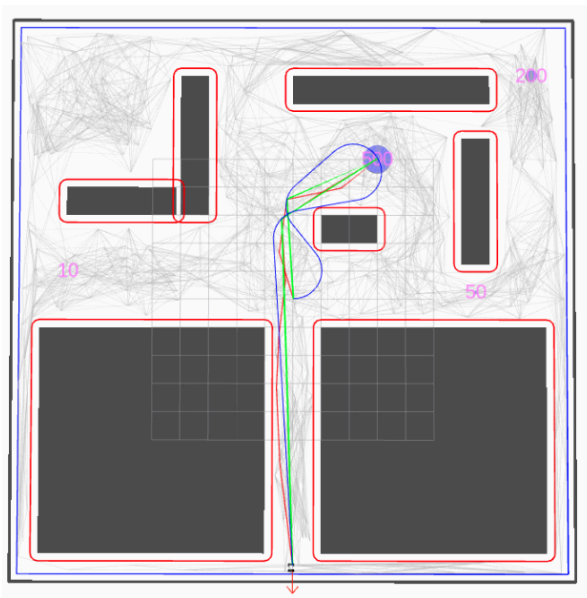
The Lyapunov Controller was updated to handle angle measurements more effectively. This prevents the robot from getting confused by 'angle jumps' (like when it switches from $180°$ to $-180°$), ensuring it follows the path smoothly without making sudden and unnecessary turns. While the high-level planner generates kinematically consistent trajectories, the controller must manage differences between the robot's actual odometry and the reference trajectory it receives from the planner. The refined implementation replaces naive error subtraction with a shortest-path angular normalization within the $[-\pi, \pi]$ range, preventing the "360-degree spin" that occurs at the $\pm\pi$ boundary. Moreover, a check was implemented for the angular control law by clamping the $\cos(e_\theta/2)$ denominator. This way we prevent the angular velocity ($\omega$) from reaching infinite values when the robot is oriented opposite to its target.

## 7. Results and Performance Metrics (optional)

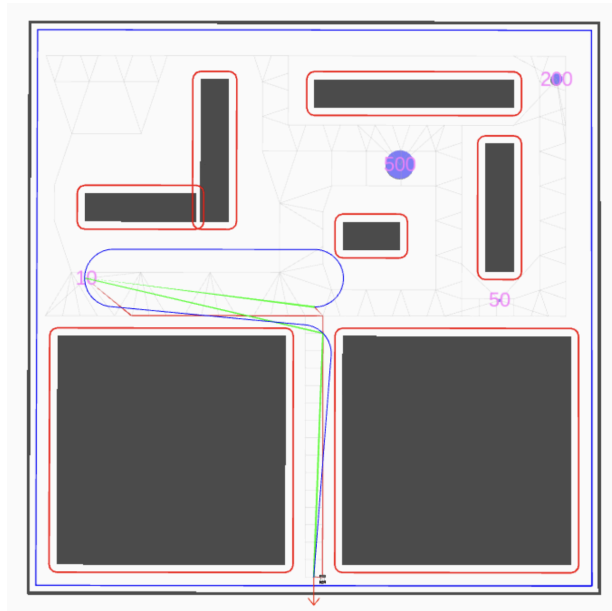
### 7.1 Analysis: Combinatorial vs Sampling Approaches

Through our simulations we have proven the respective strengths of both the combinatorial and sampling approaches.
The performance of PRM is highly dependent on the number of samples taken, where a high count leads to more reliable coverage of the space, but takes longer to calculate. Reducing the number of sampled points risks leaving uncovered areas, possibly making the path to a victim or the end gate infeasible.

ACD doesn't have this problem since the number of nodes is not an hyperparameter of the algorithm but emerges naturally from the space geometry. It places few nodes in vast open spaces, where high detail is not required, and increases the decomposition in proximity of obstacles. The lower node count can have its drawbacks as shown in the image below (ACD results). In tight enclosed spaces the resolution can be insufficient to find optimal solutions if the parameters are not satisfactory.




PRM results

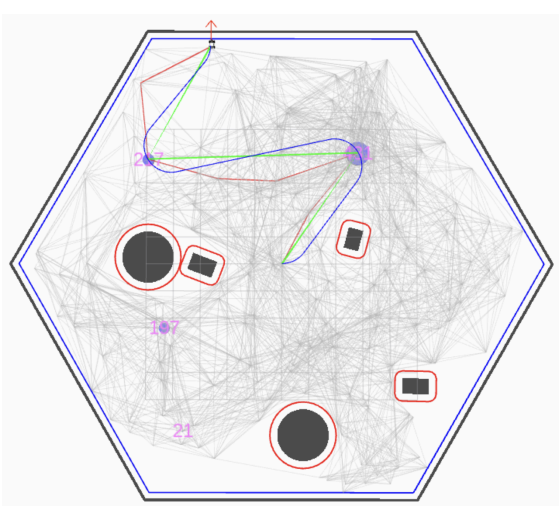



ACD results

### 7.2 Results

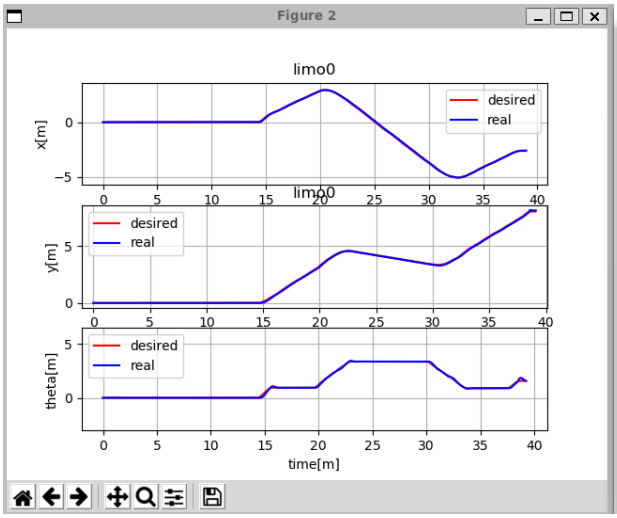
Below we provide some results of our planning project.

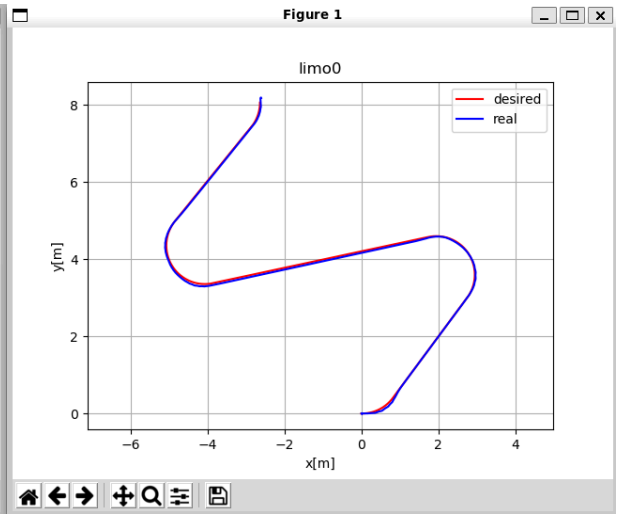
#### *PRM*



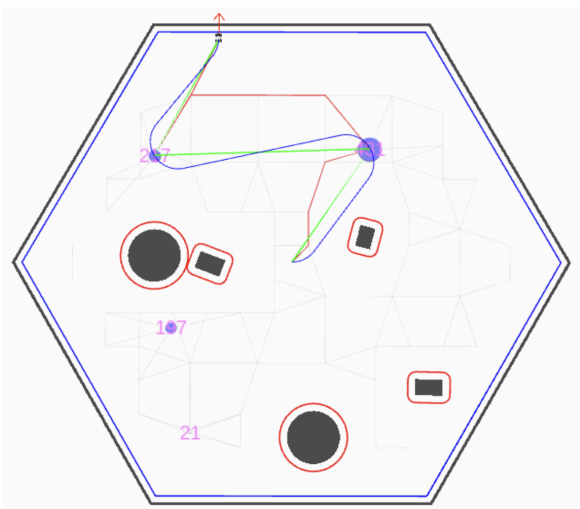

**PRM parameters:**
*n_samples*: 200
*min_connection_distance*: 2.0
*max_connection_distance*: 4.0
*resolution*: 0.05

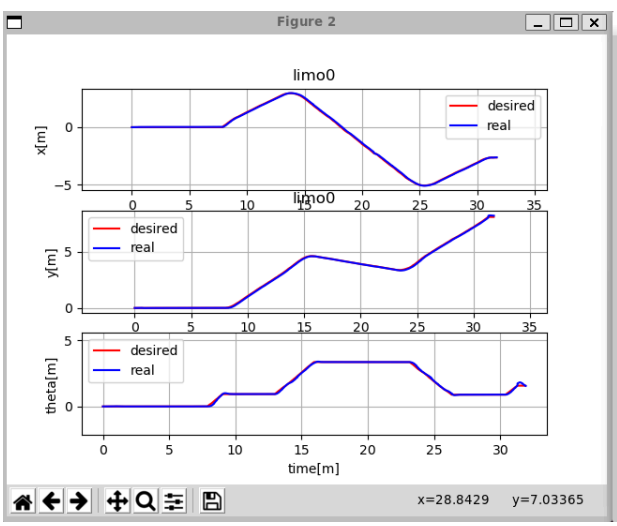
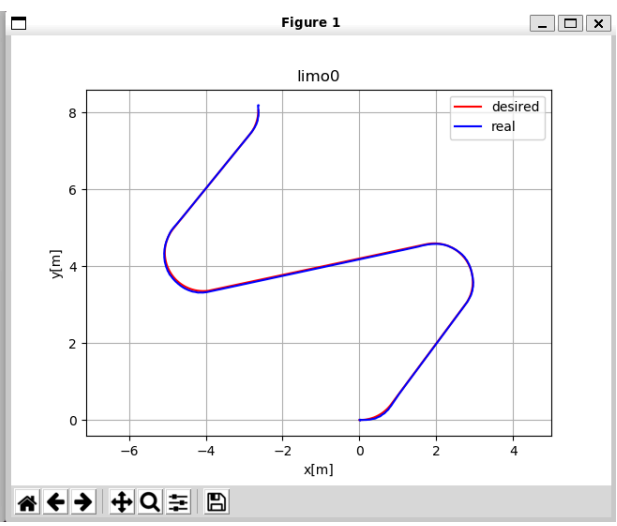
**Trajectory time:** 37.22 s




## *ACD*




**ACD parameters:**
*quadtree_max_depth*: 6
*quadtree_min_size*: 0.2
*k_victim_neighbors*: 4
*min_connection_distance*: 2.0

**Trajectory time:** 37.22 s